

CSE421: Computer Networks

Chapter 4: The Transport Layer — TCP Part I (Header, Handshaking, Termination)

Exam Study Guide

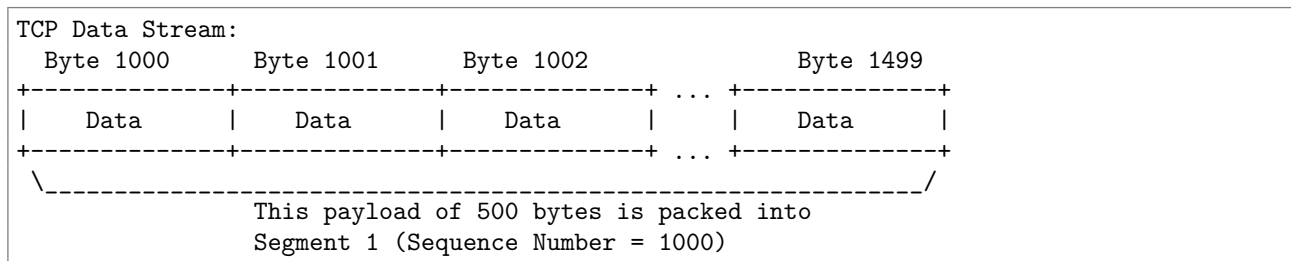
July 12, 2026

Contents

1	TCP Sequence Numbers & Initial Sequence Numbers (ISN)	2
1.1	Key Properties	2
1.2	Initial Sequence Number (ISN)	2
1.3	Calculating Sequence Numbers	2
2	Acknowledgment (ACK) Numbers	3
2.1	Formula	3
2.2	Delayed Acknowledgments	3
3	Key Header Fields: HLEN, Window Size, Options & Padding	3
3.1	Header Length (HLEN)	4
3.2	Window Size	4
3.3	Options and Padding	4
4	Control Bits (Flags)	4
4.1	The Urgent Pointer Deep-Dive	5
5	The Three-Way Handshake (Connection Establishment)	5
5.1	The Concept of the “Virtual Byte”	6
5.2	Step-by-Step Handshake Workflow	6
6	Connection Termination & Half-Close	6
6.1	Step-by-Step Termination Workflow	7
6.2	What is a Half-Close?	7

1 TCP Sequence Numbers & Initial Sequence Numbers (ISN)

TCP views the data it transmits as an ordered, continuous stream of bytes. To track and manage this stream, every single byte of data sent over a TCP connection is assigned a unique, sequential number.



1.1 Key Properties

- **Field Size:** 32 bits in the TCP header, meaning sequence numbers range from 0 to $2^{32} - 1$ (using modulo arithmetic).
- **Segment Sequence Number:** A segment can carry thousands of bytes of data. The "Sequence Number" field in the TCP header represents **the sequence number of the very first byte** carried within that specific segment.

1.2 Initial Sequence Number (ISN)

The **Initial Sequence Number (ISN)** is the sequence number assigned to the first byte of data sent in a new TCP connection.

- **How it is selected:** It is generated **randomly** for each connection, rather than starting from 0.
- **Why it must be random (Security Reason):** If the ISN always started at 0 (or followed a predictable incremental pattern), malicious attackers on the network could easily predict the sequence numbers of future packets. This would allow them to perform **Session Hijacking** or **TCP Spoofing** attacks by injecting malicious packets into an active session. Randomizing the ISN ensures connection security.

1.3 Calculating Sequence Numbers

To find the sequence number of any arbitrary segment, you can use the following recursive formula:

$$\text{Seq}_N = \text{Seq}_{N-1} + \text{Bytes}_{N-1}$$

Where:

- Seq_N : The sequence number of the current segment.
- Seq_{N-1} : The sequence number of the preceding segment.
- Bytes_{N-1} : The number of application data bytes carried by the preceding segment.

Simple Numerical Example

Suppose a client is sending a file to a server:

1. **Segment 1** starts with an Initial Sequence Number (ISN) of 124. It carries exactly 10 bytes of data.
 - *Bytes carried:* Byte 124 through Byte 133.
 - *Seq Number in header:* 124.
2. **Segment 2** is sent next. Its sequence number is calculated as:

$$\text{Seq}_2 = 124 + 10 = 134$$

- *Bytes carried:* Byte 134 onwards.

—

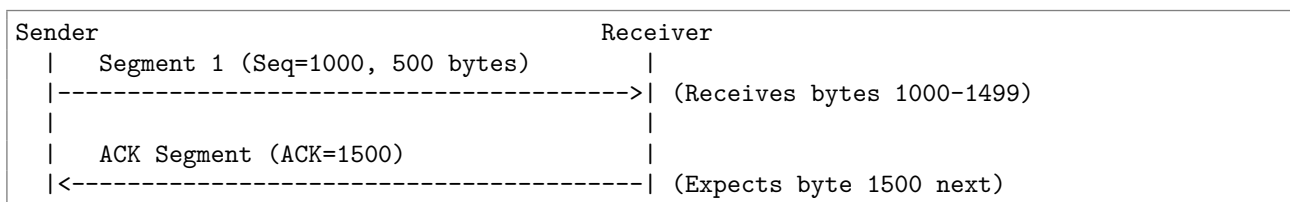
2 Acknowledgment (ACK) Numbers

TCP is a reliable protocol because of its acknowledgment mechanism. When a receiver sends an acknowledgment back to the sender, it includes an **Acknowledgment (ACK) Number**.

- **Cumulative Acknowledgment:** The ACK number is **cumulative**. It specifies the sequence number of the **next expected byte** of data from the sender.
- **Implicit Confirmation:** Sending an $\text{ACK} = X$ implicitly tells the sender: *"I have successfully received and error-checked all bytes up to $X - 1$, and I am now ready to receive byte X ."*

2.1 Formula

$\text{ACK Number} = \text{Last Received In-Order Sequence Number} + \text{Number of Bytes Received}$

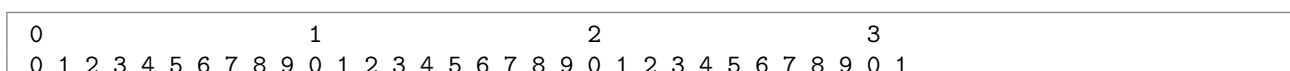


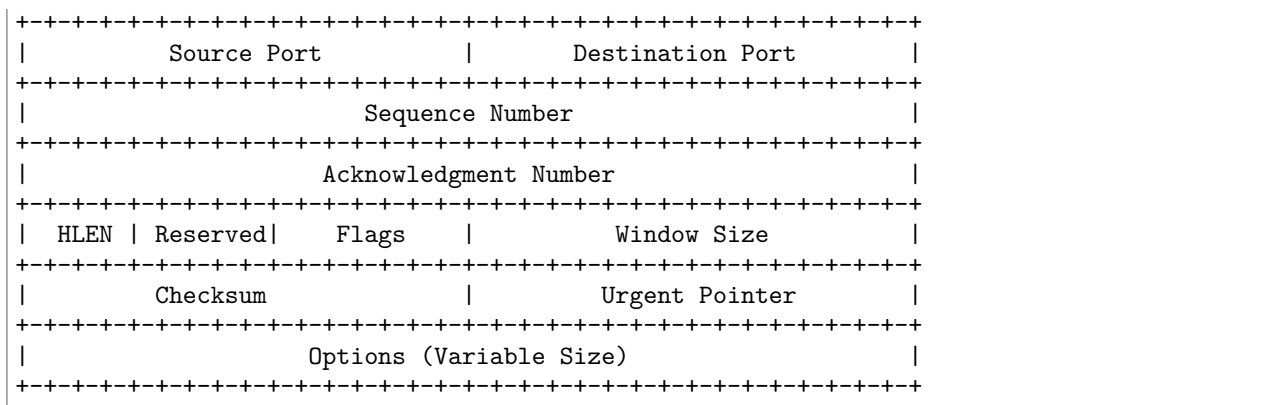
2.2 Delayed Acknowledgments

To save bandwidth and reduce network congestion, a receiver does not have to send an ACK for every single segment. It can implement **Delayed ACKs**, where it waits briefly to see if it receives more segments (or has data of its own to send back to piggyback the ACK) and then sends a single cumulative ACK covering multiple received segments.

—

3 Key Header Fields: HLEN, Window Size, Options & Padding





3.1 Header Length (HLEN)

- **Size:** 4 bits.
- **Scale Unit:** Expressed in 32-bit words (where 1 Word = 4 bytes).
- **Range:** The HLEN value can range from binary 0101 (5) to 1111 (15).
 - *Minimum Header Size (No options):* $5 \times 4 \text{ bytes} = 20 \text{ bytes}$.
 - *Maximum Header Size (With options):* $15 \times 4 \text{ bytes} = 60 \text{ bytes}$.

3.2 Window Size

- **Size:** 16 bits.
- **Purpose:** Used to implement **Flow Control** (sliding window mechanism). It advertises the buffer space currently remaining empty at the receiver side.
- **Example:** If a receiver advertises a Window Size of 200 bytes, it means: *"My receive buffer has exactly 200 bytes of free space left. Do not send more than 200 bytes of unacknowledged data, or my buffer will overflow and drop your packets."*

3.3 Options and Padding

- **Options:** A variable-length field used for parameters like Maximum Segment Size (MSS), Window Scale Factors, or Timestamps.
- **Padding:** Extra bits filled with zeros (0) added to ensure that the total TCP header size remains a multiple of 4 bytes (aligning to a 32-bit word boundary).

—

4 Control Bits (Flags)

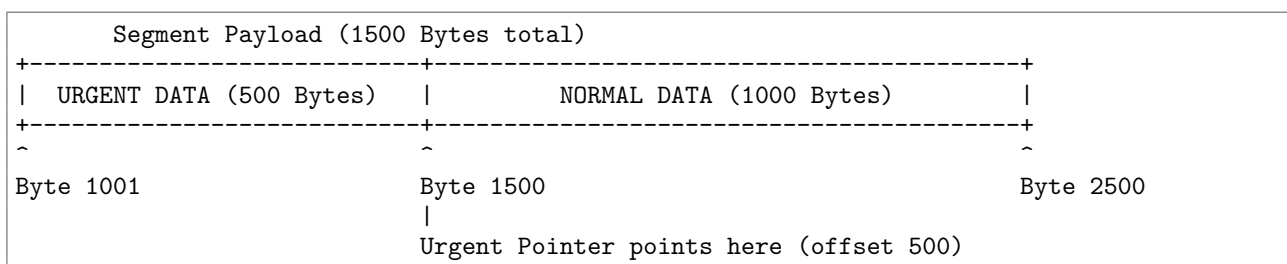
TCP uses control flags (each 1 bit) to signal the state of a segment or connection:

Flag	Name	Meaning & Description
SYN	Synchronize	Set to 1 during connection establishment (handshake) to synchronize sequence numbers.
ACK	Acknowledgment	Set to 1 to indicate the "Acknowledgment Number" field is valid. (Once connection is established, this is almost always 1).
FIN	Finish	Set to 1 when the sender has finished transmitting data and wishes to close the connection.
RST	Reset	Abruptly terminates the connection due to an unrecoverable error or host crash.
PSH	Push	Instructs TCP to immediately push all buffered data to the receiver's application layer without waiting for buffers to fill up.
URG	Urgent	Signals that this segment contains high-priority "urgent" data that must be processed before normal data.

Table 1: TCP Header Flags

4.1 The Urgent Pointer Deep-Dive

When the **URG** flag is set to 1, the receiver must look at the **Urgent Pointer** field (16 bits). This pointer indicates where the urgent data ends within the segment payload.

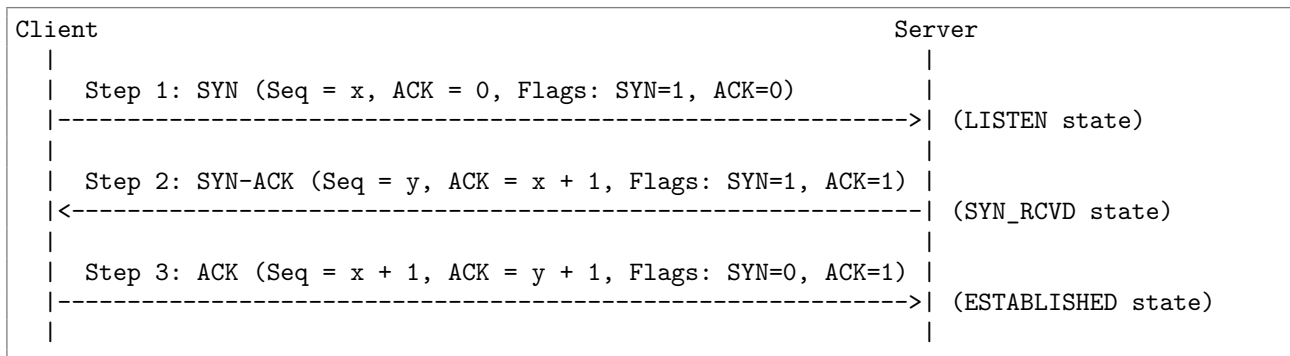


Urgent Pointer Example

- A TCP segment has Sequence Number = 1001 and carries a payload of 1500 bytes (representing bytes 1001 to 2500).
- The URG flag is set to 1.
- The Urgent Pointer value is 500.
- **Meaning:** The urgent data starts at byte 1001 (the beginning of the payload) and ends at byte $1001 + 500 - 1 = 1500$. The receiver will immediately extract and deliver bytes 1001 through 1500 directly to the application layer, skipping the normal buffer queue.

5 The Three-Way Handshake (Connection Establishment)

Before any application data can be sent, TCP must establish a logical connection.



5.1 The Concept of the “Virtual Byte”

Even though the connection-establishment segments (SYN and SYN-ACK) do not contain any physical application data, **they logically consume exactly 1 sequence number (a “virtual byte”)**. This is why the receiver increments the acknowledgment number by 1 when replying to a SYN.

5.2 Step-by-Step Handshake Workflow

1. Step 1: Client sends SYN

- *Action:* Client selects a random initial sequence number x (ISN_{client}).
- *Flags:* SYN = 1, ACK = 0.
- *Seq Number:* x .

2. Step 2: Server replies with SYN-ACK

- *Action:* Server acknowledges the client’s request by setting the acknowledgment number to $x + 1$ (consuming the client’s virtual byte). It also selects its own random sequence number y (ISN_{server}).
- *Flags:* SYN = 1, ACK = 1.
- *Seq Number:* y , *Ack Number:* $x + 1$.

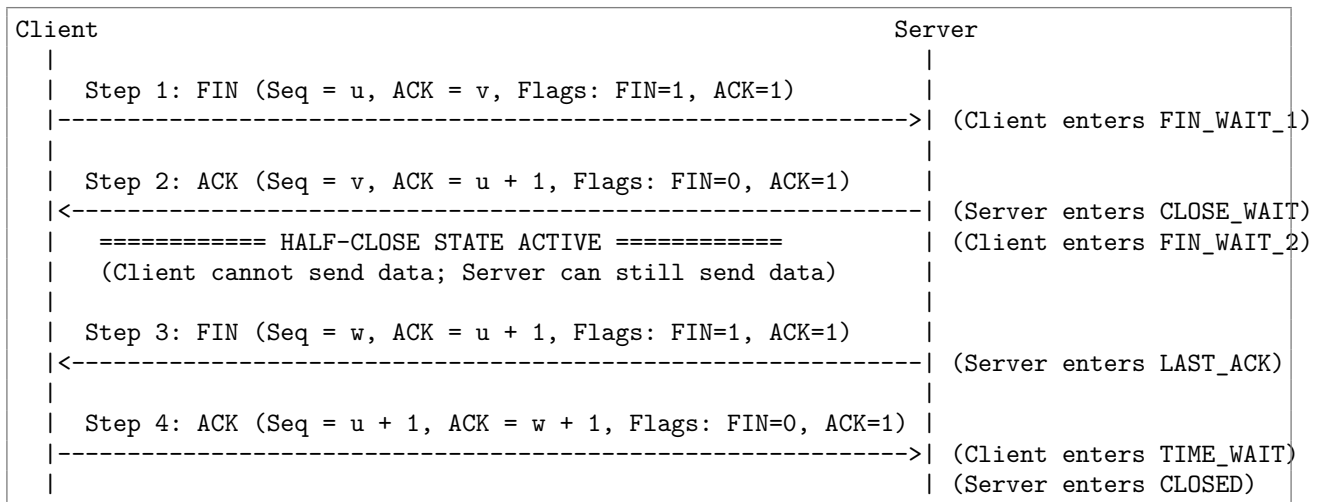
3. Step 3: Client replies with ACK

- *Action:* Client acknowledges receipt of the server’s segment by setting the acknowledgment number to $y + 1$ (consuming the server’s virtual byte).
- *Flags:* SYN = 0, ACK = 1.
- *Seq Number:* $x + 1$, *Ack Number:* $y + 1$.
- *Connection State:* Both sides are now in the ESTABLISHED state and are ready to transmit actual data.

—

6 Connection Termination & Half-Close

Closing a TCP connection is a **Four-Way Handshake** because TCP is full-duplex (meaning both directions of the connection must be closed independently).



6.1 Step-by-Step Termination Workflow

1. Step 1: Client sends FIN

- Client has finished sending all its data. It sends a segment with $FIN = 1$.
- *Seq Number: u.*

2. Step 2: Server sends ACK

- Server acknowledges receipt of the FIN by sending an ACK segment with *Ack Number* = $u + 1$.
- The client-to-server direction is now closed.

3. Step 3: Server sends its own FIN

- Once the server has finished sending its remaining data, it sends its own segment with $FIN = 1$.
- *Seq Number: w, Ack Number: u + 1.*

4. Step 4: Client sends ACK

- Client acknowledges the server's FIN by sending an ACK segment with *Ack Number* = $w + 1$.
- The client enters the `TIME_WAIT` state (waiting for a brief period to ensure the final ACK was delivered safely) before officially entering `CLOSED`.

6.2 What is a Half-Close?

The **Half-Close** occurs between **Step 2** and **Step 3** of the connection termination process.

- **Definition:** A state where one direction of the communication has been terminated (closed), but the other direction remains fully active.
- **Mechanism:**
 - The Client has sent its `FIN` and received the `ACK` from the Server. The client can **no longer send** any new application data to the server.

- However, because the Server has not yet sent its own `FIN`, the Server **can continue sending data** to the client. The client remains capable of receiving and acknowledging this incoming data.
- **Analogy:** It is like saying over a phone call: *"I am done talking, but I will stay on the line to listen to what you have left to say."*